

We find the most efficient set of bus routes and service locations that give people access to key places with the least walking and good route connections. it uses census block groups for population, yelp for service points, and gtfs data for bus routes and stops.

tool 1 collects service data. the script reads block group shapes and finds the centroid of each group. it calls the yelp api for several categories such as grocery, pharmacy, restaurant, and clinic. for each centroid it stores a few nearby businesses with name, rating, latitude, longitude, and category. the result is a csv file called `milwaukee_multiservice_yelp.csv`. each row represents one business near a block group. this file shows where important services are located across the study area.

tool 2 cleans and links data. it removes duplicates in the yelp csv and keeps one record per unique business. it reads gtfs text files from the transit system: `routes.txt`, `stops.txt`, `trips.txt`, and `stop_times.txt`. it connects routes to their stops and finds where routes overlap or share stops. It then measures distances between block group centroids, stops, and services. it creates three output files. `w.csv` lists which blocks can walk to which routes within a set distance and records the minimum distance in meters. `t.csv` lists which blocks can reach which services by walking to a route and traveling up to a limited number of transfers. `route_neighbors.csv` lists which routes share nearby stops and are considered connected. these files turn the raw spatial data into simple tables that describe coverage and reachability.

tool 3 performs the optimization. it reads `w.csv`, `t.csv`, the cleaned services file, and `route_neighbors.csv`. it builds a mixed integer linear model that chooses a set of routes and services. the model tries to cover as many populated blocks as possible, penalizes long walks, and rewards routes that connect to each other. constraints limit how many routes and services can be chosen. the output files are `selected_routes.csv` with the chosen route ids, `selected_services.csv` with the chosen service locations and their coordinates, and `block_assignments.csv` showing which route each block group is assigned to. the file `solution_summary.txt` records the solver status and objective value. these results identify the combination of routes and services that gives the best overall access under the limits.

tool 4 summarizes and maps the results. it joins the block assignments with the shapefile to create `assigned_blocks.geojson` for mapping. it extracts the selected services and saves them as `selected_services.geojson`. it computes key numbers such as average walking distance, number of covered blocks, and category counts of selected services. the file `solution_summary_readable.txt` lists these metrics in plain text. these outputs help visualize and interpret what the optimization achieved.

tool 5 prepares data for later time-based analysis. it reads the full gtfs directory and the list of selected routes. it filters `routes.txt`, `trips.txt`, `stop_times.txt`, and `stops.txt` to include only those routes and their related trips and stops. it writes them to a smaller folder called `gtfs_subset`. this folder contains only the data needed for the chosen network, which makes later analysis faster.

The next step is to add time to the model. gtfs provides departure and arrival times for each stop. future scripts can compute how reachable each service is at different times of the day. one method is to run a travel time calculation every 30 seconds through a 24-hour period. this would

build a time matrix showing how accessibility changes during the day. combining this with the current spatial optimization will show not only which routes cover the most people but also when those routes provide good service.

Optimization Model

B = set of block groups from W.csv and T.csv (column GEOID).

R = set of routes from W.csv (column route_id).

S = set of services from milwaukee_yelp_services_clean.csv (row index service_idx).

W-pairs = $\{(b,r)\}$ from W.csv with distance $d_{br} = \min_dist_m$.

T-pairs = $\{(b,s)\}$ from T.csv.

N(r) = neighbor routes of r from route_neighbors.csv.

p_b = population weight for block b (if not provided, use 1).

R_max, S_max = budgets.

alpha, beta, lambda, mu = objective weights.

Decision Variables

$x_r \in \{0,1\}$ selects route r.

$y_s \in \{0,1\}$ selects service s.

$z_{br} \in \{0,1\}$ assigns block b to route r if $(b,r) \in W$.

$u_b \in \{0,1\}$ indicates block b has any selected route.

$w_{bs} \in \{0,1\}$ links block b to service s if $(b,s) \in T$.

$v_b \in \{0,1\}$ indicates block b has at least one reachable selected service.

$q_r \in \{0,1\}$ indicates route r is connected to at least one other selected route.

(Looks at next page for function and constraints)

Objective Function

$$\begin{aligned}
& \text{maximize} && \sum_{b \in B} p_b (\alpha u_b + \beta v_b) - \lambda \sum_{(b,r) \in W} d_{br} z_{br} + \mu \sum_{r \in R} q_r \\
& \text{subject to} && z_{br} \leq x_r && \forall (b,r) \in W \\
& && u_b \leq \sum_{r: (b,r) \in W} z_{br} && \forall b \in B \\
& && \sum_{r: (b,r) \in W} z_{br} \leq 1 && \forall b \in B \quad (\text{optional single assignment}) \\
& && w_{bs} \leq y_s && \forall (b,s) \in T \\
& && v_b \leq \sum_{s: (b,s) \in T} w_{bs} && \forall b \in B \\
& && \sum_{r \in R} x_r \leq R_{\max} \\
& && \sum_{s \in S} y_s \leq S_{\max} \\
& && q_r \leq x_r && \forall r \in R \\
& && q_r \leq \sum_{r' \in N(r)} x_{r'} && \forall r \in R \\
& && x_r, y_s, z_{br}, u_b, v_b, w_{bs}, q_r \in \{0, 1\}
\end{aligned}$$

Files I've used for this:

d_{br} comes from min_dist_m in $W.csv$.

B and R come from the GEOID and route_id columns in $W.csv$.

T defines the valid (b,s) pairs using service_idx consistent with $\text{milwaukee_yelp_services_clean.csv}$.

$N(r)$ is built from $\text{route_neighbors.csv}$ and activates q_r and the μ term.

If a population csv is not supplied, set $p_b = 1$.

Results:

- The optimization used the racine transit feed and produced an optimal result.
- all nine available bus routes were selected along with thirty-five reachable service locations.
- the chosen routes together covered all ninety-six block groups that had walking access to at least one bus stop.
- every populated area in the study region can reach at least one selected route and one selected service through the optimized network.

- the solver completed successfully and confirmed that the data and constraints were consistent.
- the average walking distance from a block centroid to its assigned stop was a few hundred meters, within the eight-hundred-meter limit used in the model.
- the selected services represented a mix of categories such as grocery, clinic, and restaurant, so access was spread across multiple types of destinations.
- the nine chosen routes formed a connected set according to the route_neighbors file, meaning riders can transfer easily between lines.
- the main output files include:
 - selected_routes.csv, listing the nine chosen route ids
 - selected_services.csv, listing the thirty-five selected service points with names and coordinates.
 - block_assignments.csv, linking each block group to its assigned route
 - solution_summary.txt, recording the solver status and final objective value
 - assigned_blocks.geojson and selected_services.geojson, which can be opened in a gis viewer to show the spatial coverage